

bare/data/obj-a7f2b3.sh

sync/node/lock-e9a1.conf

crypt/path/valid-5d7c.json

Architecting sg-vault

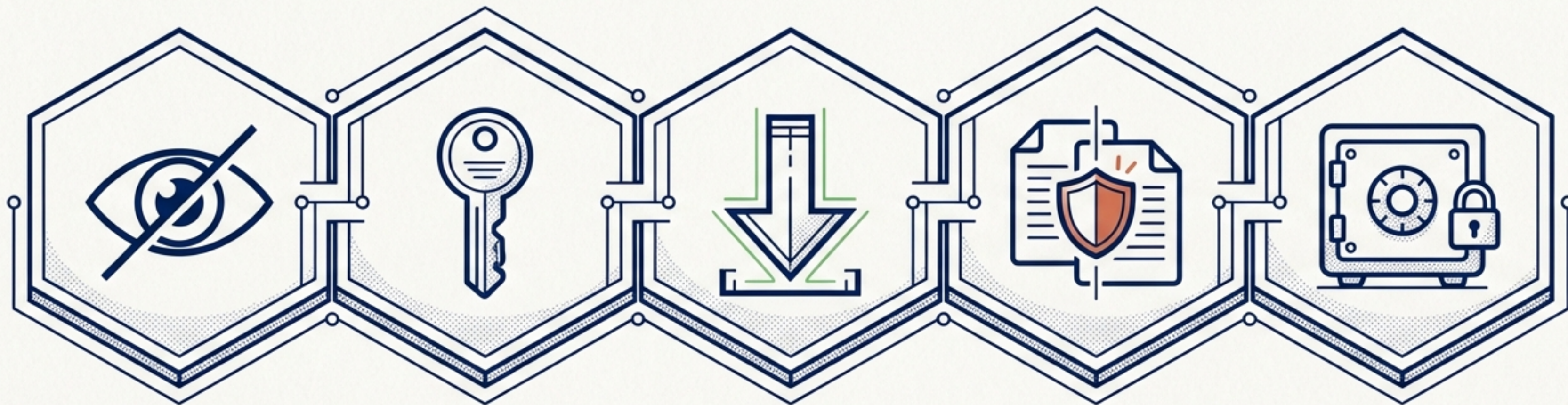
The Unified Storage and Cryptographic Branch Model

v0.13.30 — Validated via Simulation v6.
For Engineering, DevOps, and AppSec Review.

sync/node/lock-e9a1.conf

secure/vault/entry-0x4a.log

The Physics of the Vault



Zero Knowledge

The server stores only opaque encrypted blobs. No filenames, no branch names, no relationships visible.

Cryptographic Identity

Provenance lives in the graph, not server policy. Every branch has a key pair; every commit is signed.

Remote Authority

The remote named branch takes precedence. You must accept remote commits before pushing.

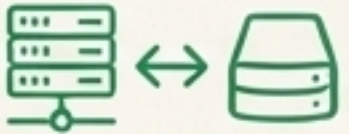
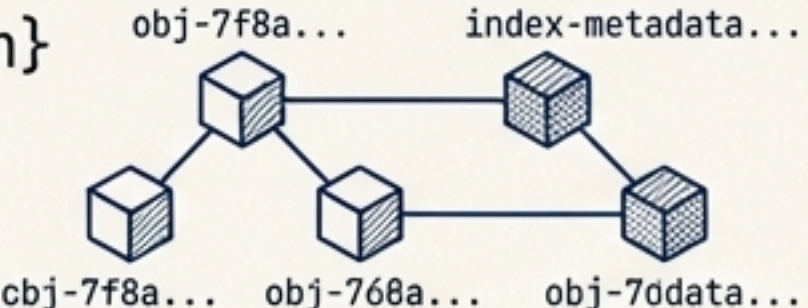
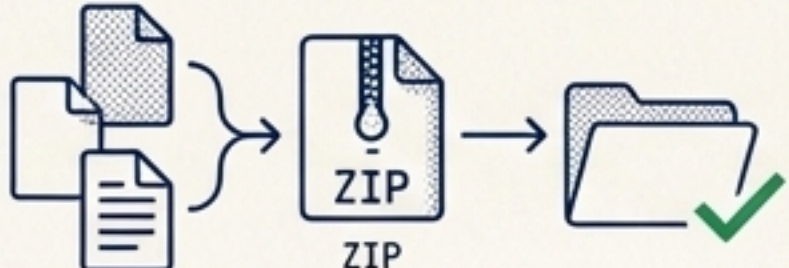
Isolated Conflicts

The named branch never has conflicts. The clone branch absorbs all conflicts for local resolution.

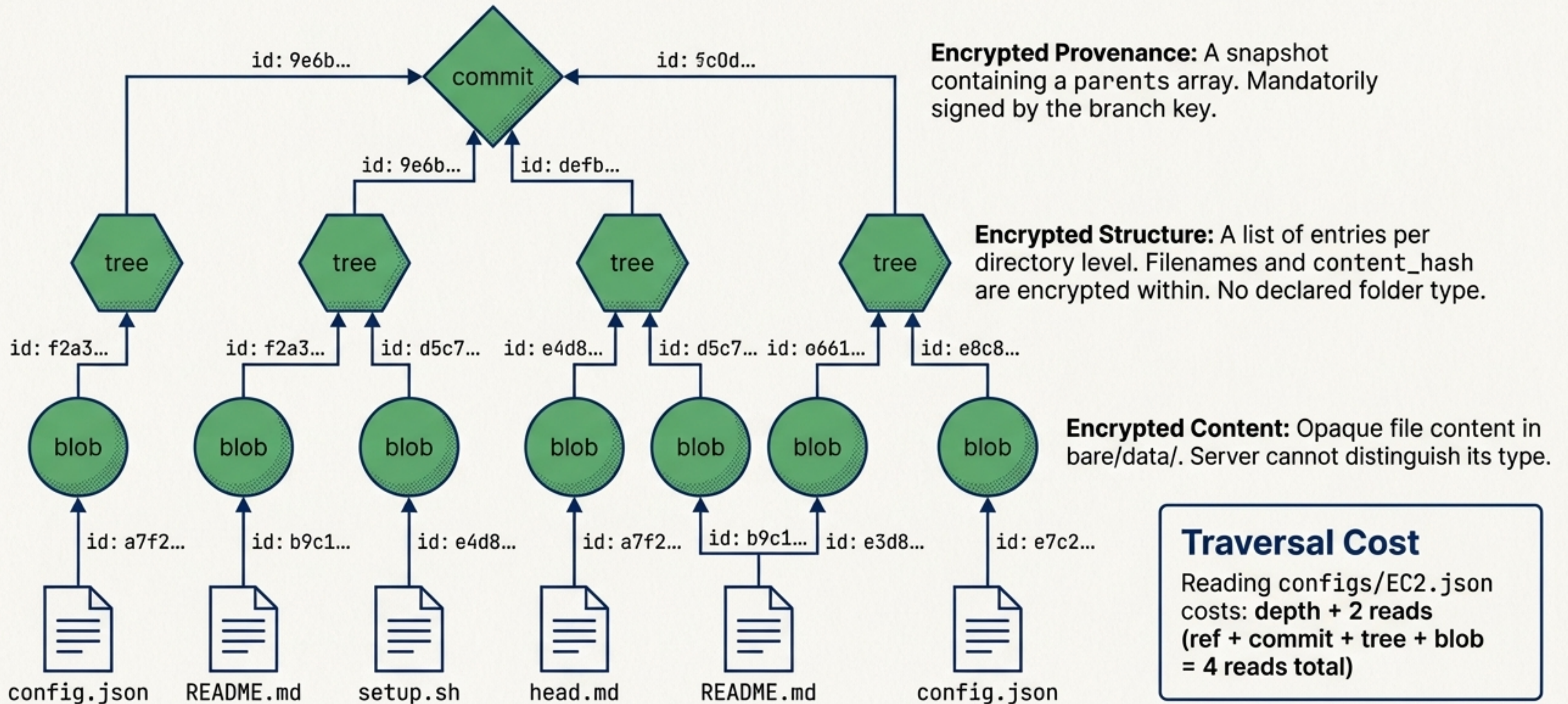
Portable Bare Vault

The bare/ directory is the complete vault unit. It syncs, packs into a zip, and survives deletion of either side.

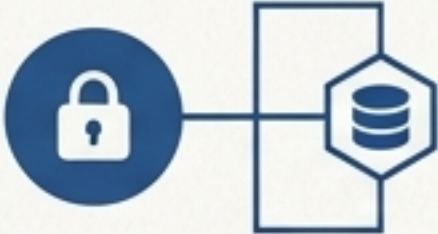
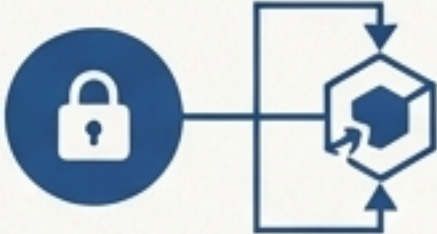
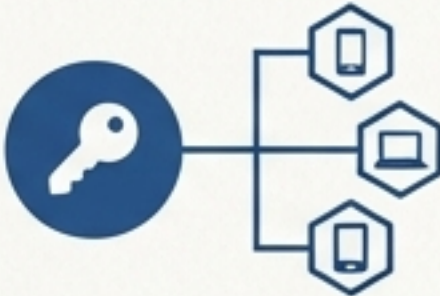
The Two Enclaves: Portable Vault vs Device State

	 bare/	 local/
Purpose	The complete, self-contained portable vault.	Device-specific execution and configuration state.
Network Behaviour	Identical on server and local disk; safely synced. 	Never leaves the device; never synced. 
Key Contents	Opaque obj-{hash} files grouped by type, encrypted indexes. 	Vault key, branch_key.pem, remote tracking state. 
Portability	Can be packed as a zip and fully restored independently. 	Re-generated on a fresh clone. A bare clone produces no local files. 

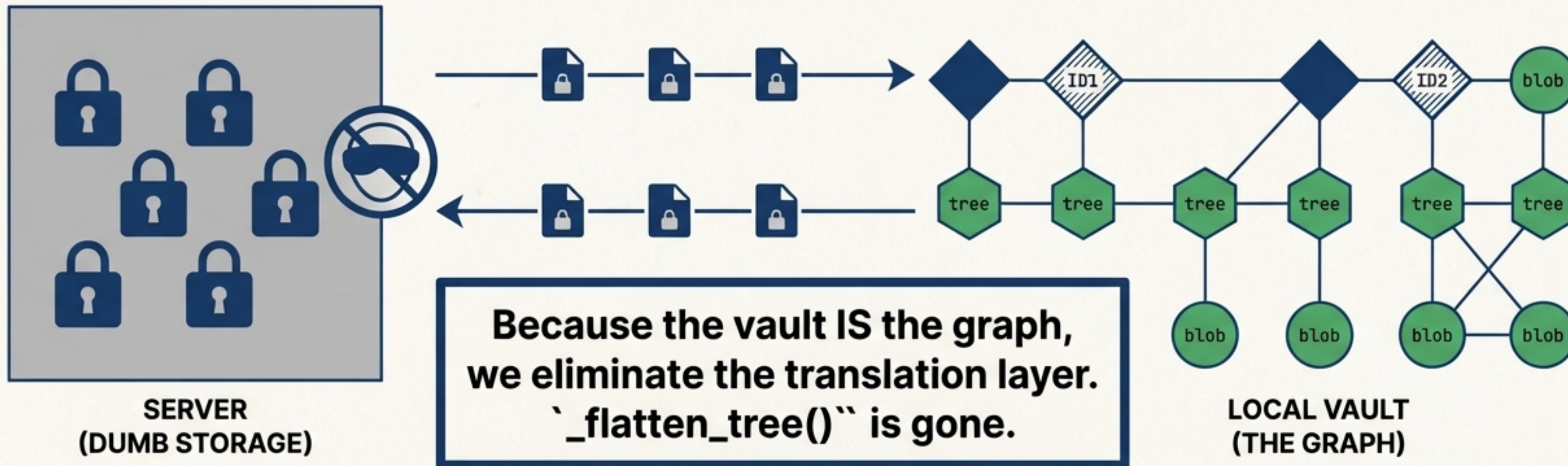
The Opaque Object Graph



Branch Identities and Cryptographic Scopes

		
Clone Branch Key (EC P-256)	Named Branch Key ('current')	User's Main Key
Location: <code>local/branch_key.pem</code>	Location: Inside the vault, encrypted with vault read_key.	Location: <code>~/.sg-send/</code> or external keychain.
Mobility: NEVER leaves the device.	Mobility: Synced inside bare/	Mobility: Multi-device.
Role: Provenance for a specific device/session. Push always succeeds; isolated to the user.	Role: Authoritative integration point. ONLY receives merges; mathematically rejects direct commits.	Role: Unlocks locked named branches or administrative actions.
Loss Consequence:  Branch is permanently locked (a normal lifecycle event).	Loss Consequence: N/A (Synced within vault)	Loss Consequence: Re-import from backup.

sg-vault is a Zero-Knowledge, Cryptographic Git



Natural Delta Push

- Change one file, and only the new blob and its specific sub-tree path are uploaded. No complex diff algorithms required.

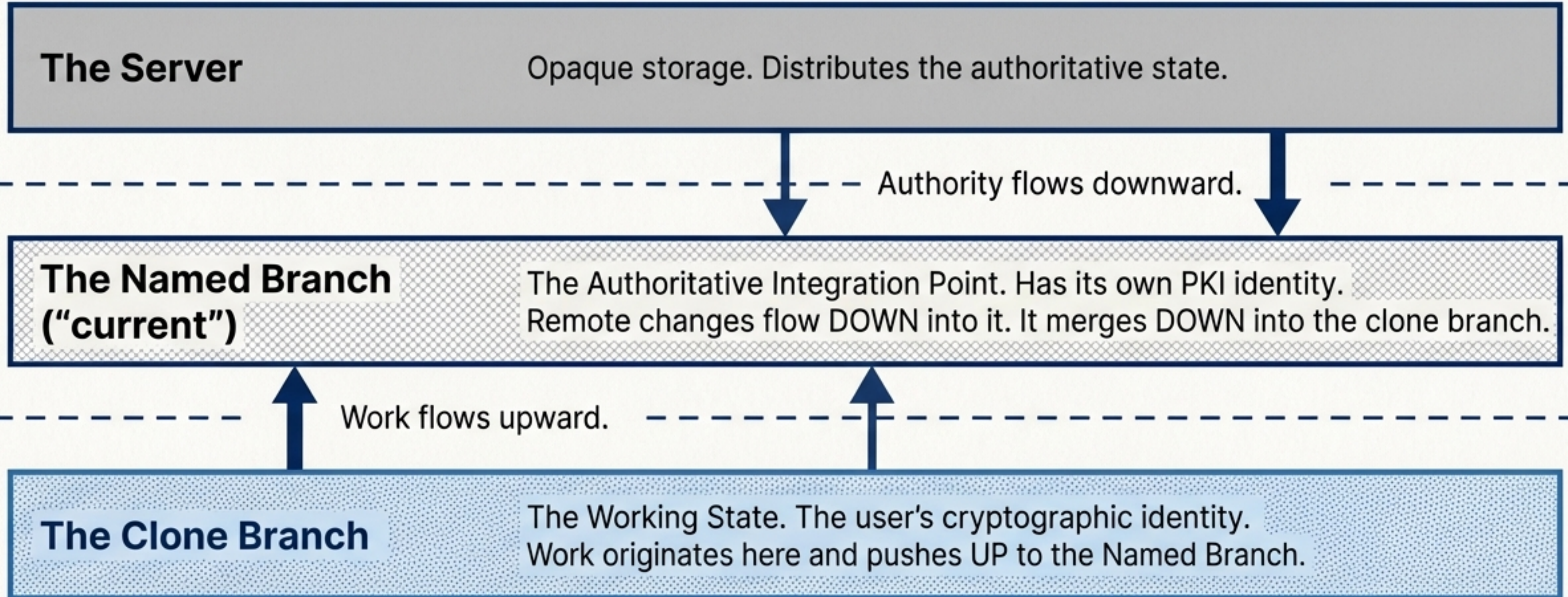
Automatic Deduplication

- Identical sub-trees across branches automatically share the same content-addressed objects.

Immutable Provenance

- The server knows nothing about the data, yet the data mathematically proves its own history through signature chains on every commit.

The Three-Layer Workflow Engine

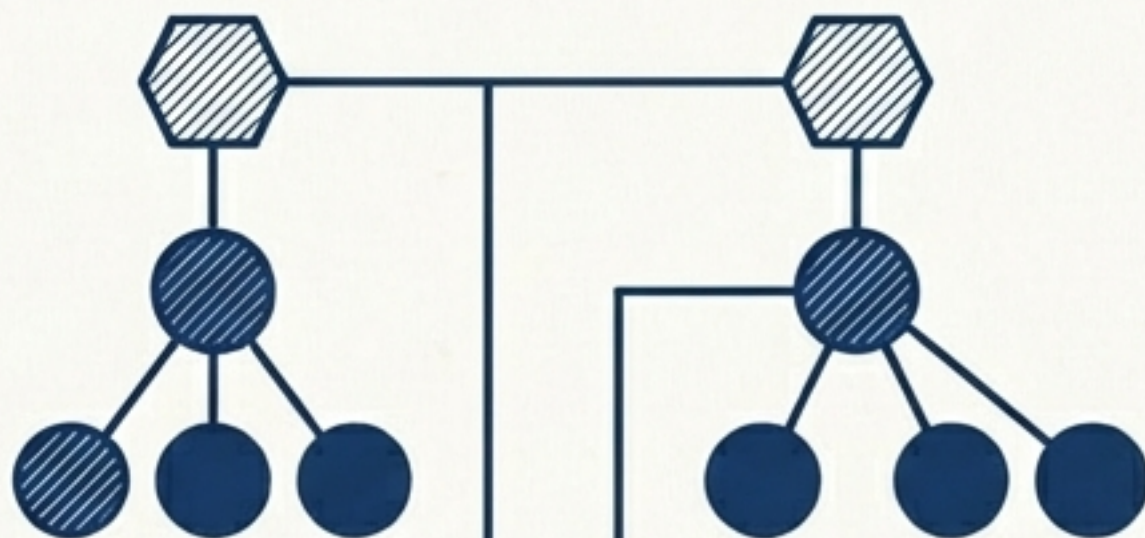


Rule Callout

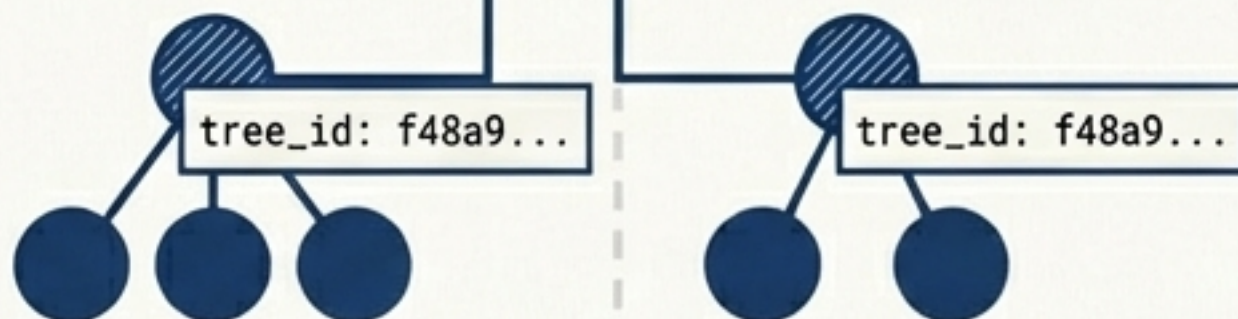
Direct commits to the Named Branch are mathematically rejected.

The Sub-Tree Shortcut Mechanism

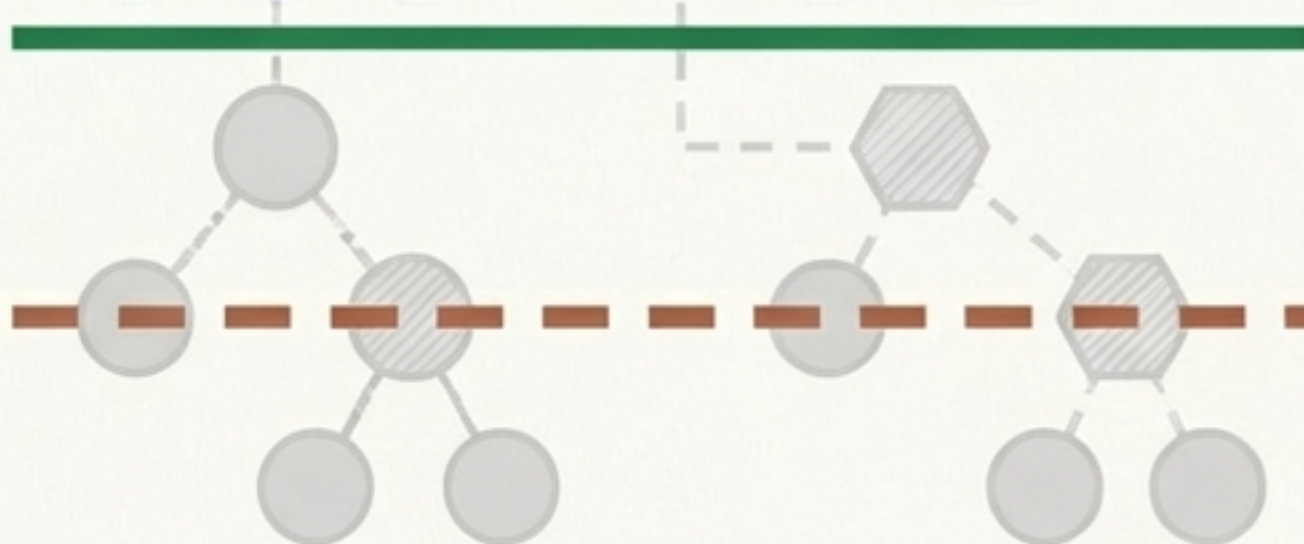
**Server &
Named Branch
(Authoritative)**



**Merge
Comparison**



SHORTCUT SHIELD



**Skipped
Sub-Tree**

- **The Process:**
During a three-way merge, the algorithm compares `tree_id` values at each node level.
- **The Shortcut:**
If a sub-tree ID matches, the system knows the entire sub-folder is mathematically identical.
- **The Result:**
The merge instantly short-circuits. It skips reading any individual files within that folder, allowing massive vaults to merge in milliseconds.

The Gravity of Conflicts

The Server

Fast-Forward: Zero Conflicts

The Named Branch
("current")

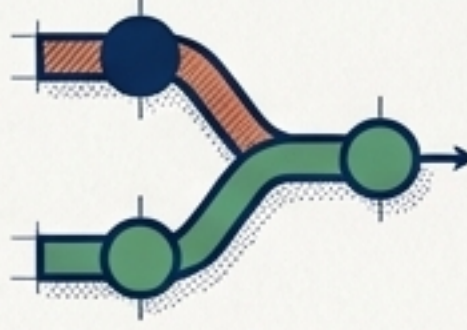
The Clone Branch

The .conflict Convention

- Conflicts are heavy; they fall to the bottom layer. The named branch is mathematically shielded.
- When divergence occurs (e.g., README.md), the remote named branch version wins by default.
- The local working version is preserved as README.conflict.md.

 **Push Blocked:** You cannot push as long as any .conflict file exists in the working directory.

Integration Rules: Fast-Forward vs. Merge

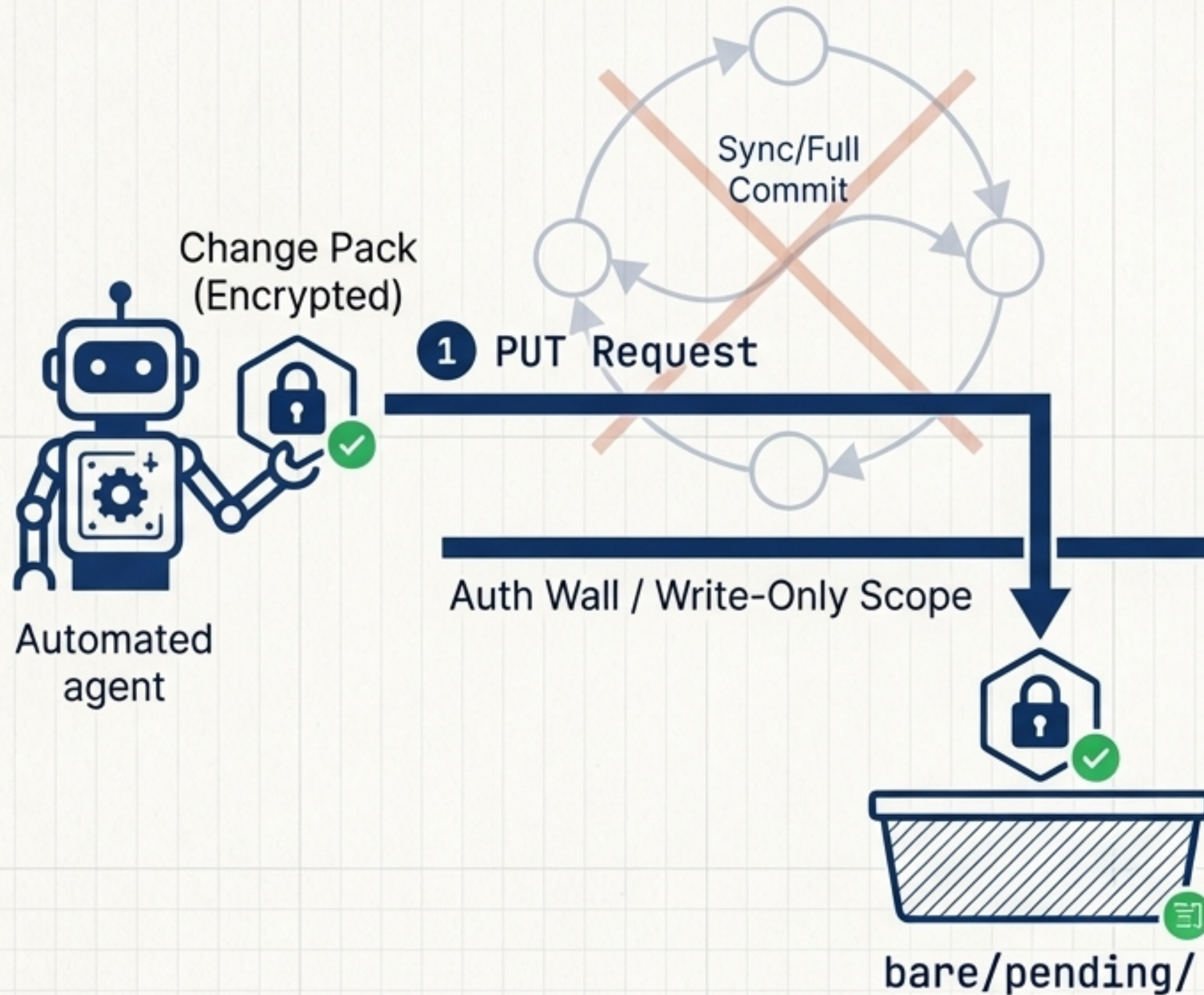
Operation	Action	Why
Local Named Branch ← Remote	FAST-FORWARD 	The local named branch has no independent commits. It simply moves the pointer to match the remote authority .
Named Branch ← Clone Branch (Push Step 6)	MERGE COMMIT 	Preserves the boundary of “this was integrated into the named branch”. Must be signed by the Named Branch key.
Clone Branch ← Named Branch (Pull)	MERGE COMMIT 	Preserves the history of pulling remote changes. Signed by the Clone key. <u>This is where conflicts surface.</u>

Network Efficiency and Request Budgets

Zero-Cost Operations (Entirely Local) init, commit, merge into named branch, status.	Network Cost  0 API calls
Standard Sync (Push/Pull)	 1 GET + 1 POST Utilises the new Batch API to fetch the ref, then upload/download all required objects in a single internet round trip.
Heavy Operations clone (no cache, 25 files).	 ~30 GETs + 1 POST Requires walking the tree + registering the branch.

Insight: The architecture fundamentally shifts work from network I/O to local CPU (cryptography), optimising for high-latency environments.

The Keyless Change Pack



The Use Case:

High-frequency writes from external systems that cannot afford a full commit ceremony (which usually requires 5+ network requests).

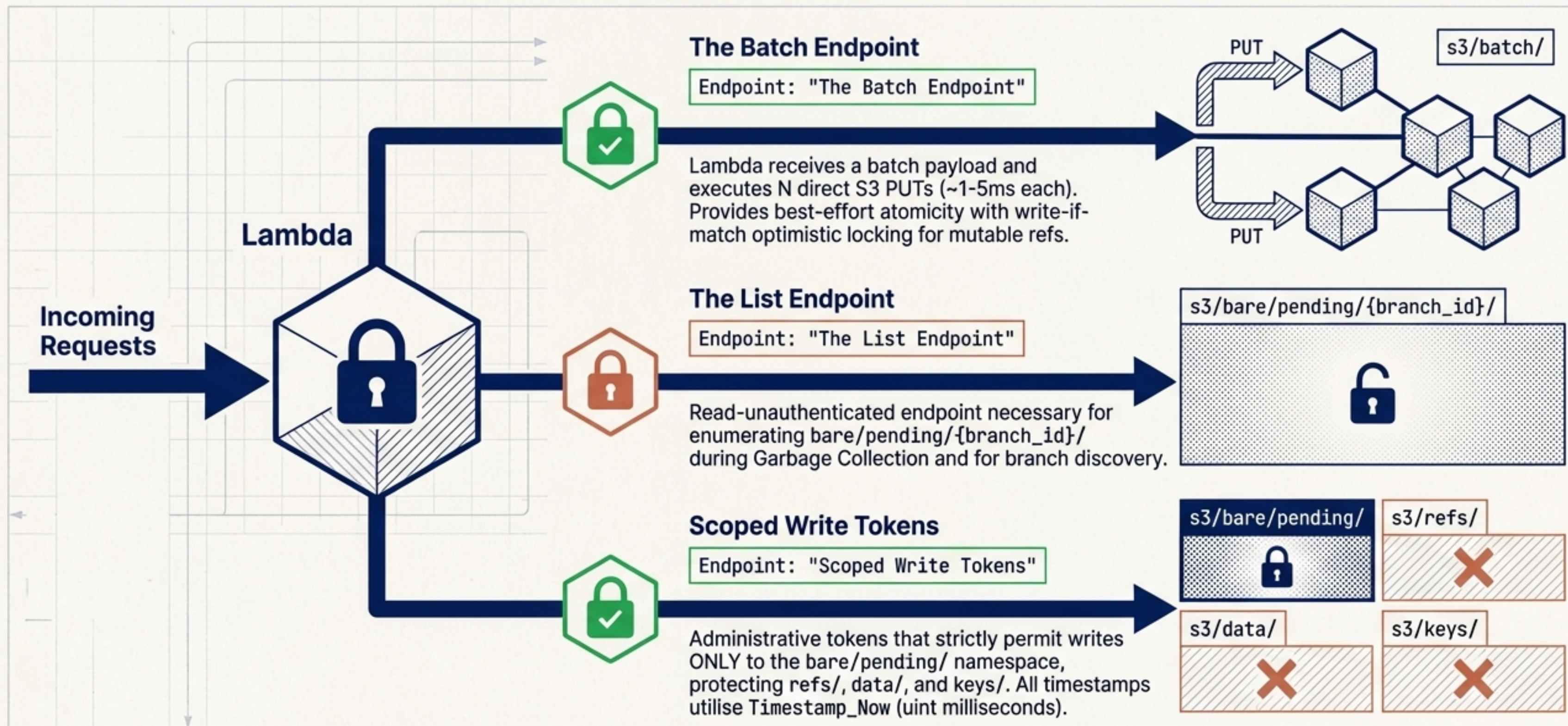
The Mechanism:

The entity has never cloned the vault and has no read access. Using the branch public key, its own signing key, and a scoped write-only token, it executes exactly 1 PUT request.

The Result:

The vault becomes an append-only target. The server stays dumb and does not verify signatures; the Garbage Collector (GC) verifies the encrypted payload upon draining the pending bucket during the next sync.

The Execution Backbone: New API Surface



Implementation Roadmap: Zero Flag Days

Phase 1: Read Both, Write Both

Read legacy `tree.json` to build new sub-trees. **Write** new sub-trees AND generate legacy `tree.json` for UI backward compatibility.

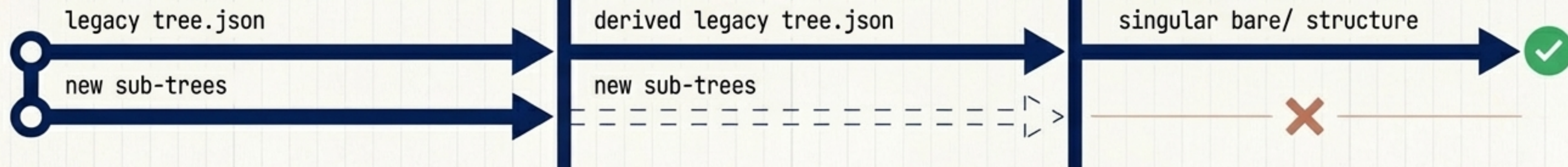
Phase 2: Derive Legacy



Server generates `tree.json` on the fly. It transitions from primary storage to derived data. `tree_to_flat()` replaces `_flatten_tree()`.

Phase 3: Remove Legacy

`_flatten_tree()` and `_build_tree_json()` are deleted. The bare/ structure becomes the singular format.



Takeaway: Each phase is independently deployable and verifiable. Non-destructive migration.